BIG DATA 301

9:30 AM.

ADVANCED HIVE CONCEPTS

MANAGING DATA THROUGH PARTITIONS

Partitioning enhances query performance by splitting Hive tables into manageable parts. It is based on single or multiple columns.

Benefits:

- Improves performance: Only relevant partitions are scanned, avoiding Full Table Scan (FTS).
- Storage: Data is stored in subdirectories within the table directory.

```
CREATE TABLE Songs_data_sp <---- Partitioned table name
(song_id STRING,
artist_id STRING,
album_name STRING,
song_views INT)
PARTITIONED BY (rating FLOAT) <---- Partitioned column name
ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n';
```

Types:

➔ Static Partitioning
- ◆ Creation: Based on static value provided by the user.
- ◆ Data Insertion: Explicitly inserted into the corresponding partition.
- ◆ Alteration: Can be altered later.
- ◆ Challenge: Difficult with diverse column values; requires separate INSERT statements.

```
INSERT OVERWRITE TABLE Songs_data_sp <-- Partitioned table name
PARTITION (rating = 4.3) <-- Partition value
SELECT Song_track_id, artist_id, album_name, song_views
FROM Songs_data where rating = 4.3;        } Select query
```

➔ Dynamic Partitioning

- ◆ Creation: Hive creates partitions based on distinct values of the partitioned column.
- ◆ Example: In a Songs dataset, the rating column (0.0 to 5.0) creates partitions for all distinct values.
- ◆ Query: Only partitioning columns are provided.
- ◆ Alteration: Cannot be altered later.

```
INSERT OVERWRITE TABLE Songs_data_dp <-- Partitioned table name
PARTITION (rating) <-- Partition column
SELECT song_track_id, artist_id, album_name, song_views, rating
FROM songs_Data;                           } Select query
```

Viewing Partitions: SHOW PARTITIONS <PartitionalTableName> command.

Limitations

- Partition Column Selection: Poor choice can lead to many small HDFS directories and files.
- Example: Few records for ratings 0.0 to 1.0 create unnecessary folders, affecting Namenode and cluster performance.

SAMPLING DATA USING BUCKETS

Another technique for decomposing data sets into more manageable parts. Data is distributed to each bucket based on the bucketed column value. The required number of buckets need to be specified while creating the table.

Optimizes Select query. **Hash function** is used to distribute the data among the buckets. Applied on both tables and partitions.

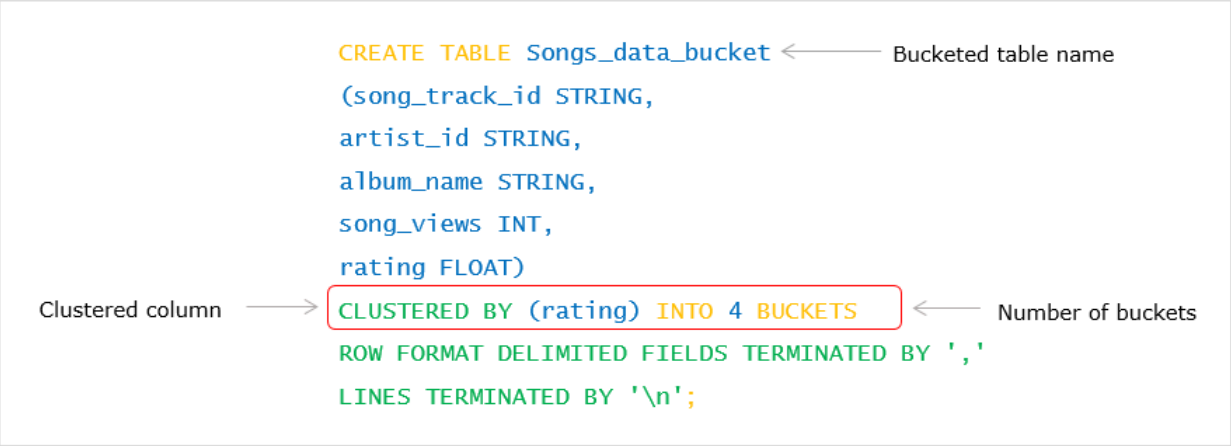
A table can have multiple partitions, and a partition can have multiple buckets.

A bucket of a table is represented in HDFS as a file inside a table or partition directory.

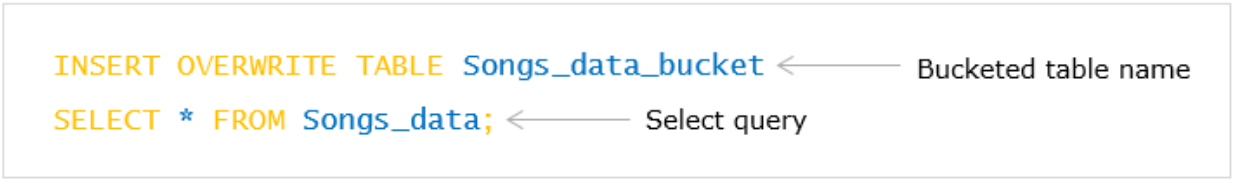
2 reasons for using buckets:

- Helpful in data sampling
- Map side joins are executed better with bucketed tables

Limitation: Bucketing does not ensure that data is evenly distributed across buckets.



The below *INSERT* command creates *4* files inside *Songs_data_bucket* directory.

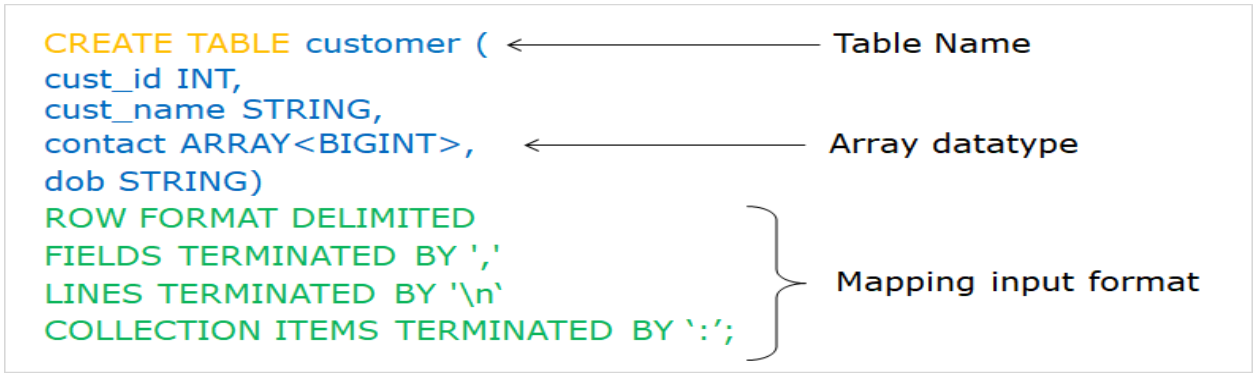


HIVE COMPLEX DATA TYPES

Allow multiple values within a single column.

Array

Stores ordered list of homogenous data.



Access specific array datatype: SELECT contact[1] FROM customer;

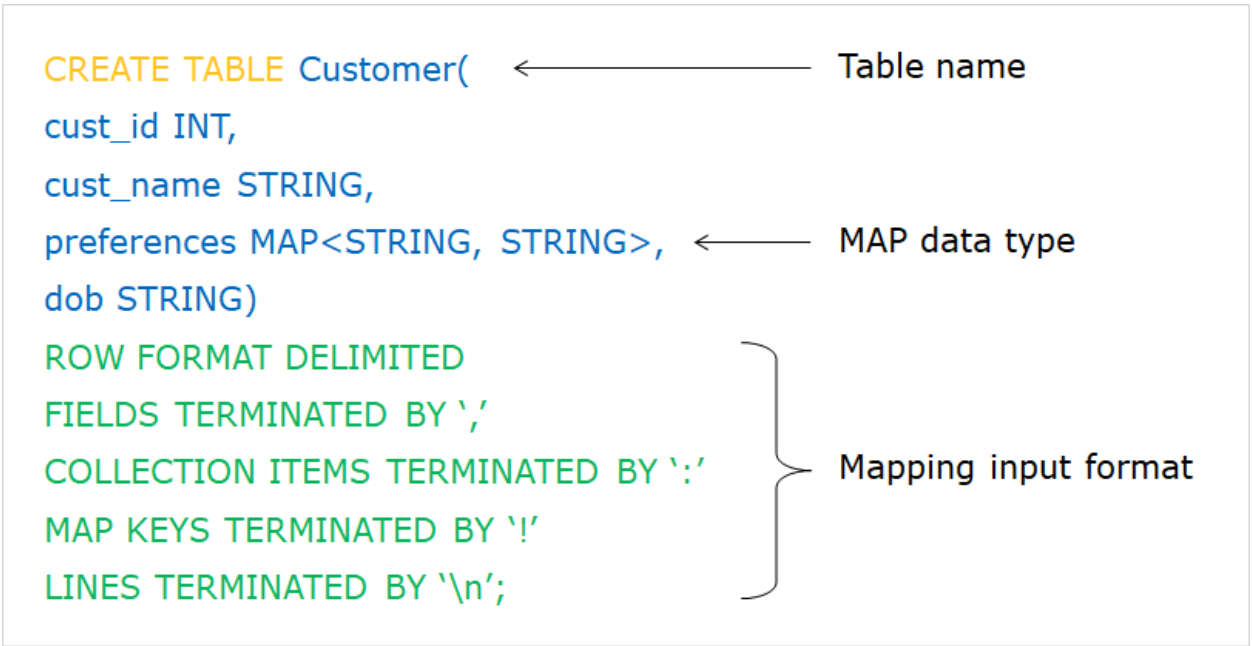
Best practices:

- Use array when number of elements are fixed;
- When there are no frequent checks and comparisons made on the elements;

Map

Stores a collection of key-value pairs. Values can be accessed using keys.
Defining map datatype requires:

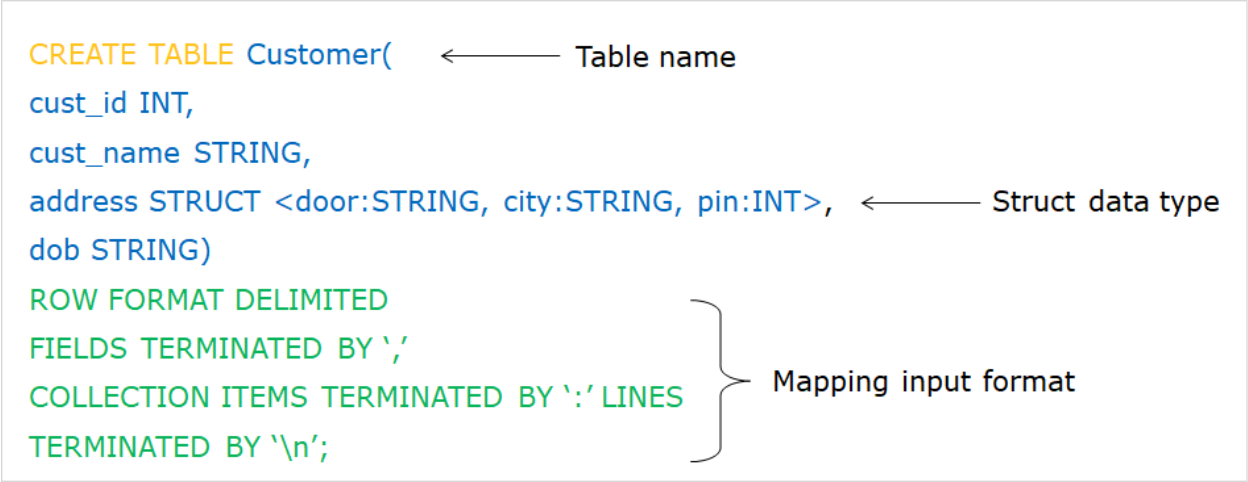
- ➔ Collection item delimiter
  - ◆ Special character which delimits one key-value pair from another
  - ◆ Uses COLLECTION ITEMS TERMINATED BY clause
- ➔ Map key delimiter
  - ◆ Special character which delimits key and value
  - ◆ Uses MAP KEYS TERMINATED BY clause in Create Table statement



SELECT preferences['genre'] FROM customer;

Struct

Stores sets of fields of different data types.



SELECT address.door FROM customer;

HIVE FUNCTIONS

Built to perform some operations like logical, relational and mathematical on the datasets and tables.

2 categories of functions:

- Built-in
- User Defined

Built-in functions

Predefined and readily available to use.

String functions: to manipulate and transform strings.

Function	Return Type	Description	Sample
LTRIM(String Z)	String	It fetches and return string without leading spaces (left-hand side of string)	LTRIM(' String') results in 'String'
RTRIM(String Z)	String	It fetches and return string without leading spaces (right-hand side of string)	RTRIM('String ') results in 'String'
SUBSTR(String, <len>)	String	It fetches and return a part of string from start position to the specified length	SUBSTR('New Hive Advanced', 5, 4) results in 'Hive'

Mathematical functions:

Function	Return Type	Description	Sample
ROUND(DOUBLE Z)	DOUBLE	It returns the rounded value of the double Z	ROUND(10.5) results in 11
CEIL(DOUBLE Z)	DOUBLE	It returns the minimum BIGINT value which is equal to greater than Z	CEIL(10.3333) results in 11
FLOOR(DOUBLE Z)	DOUBLE	It returns the maximum BIGINT value which is equal to greater than Z	FLOOR(10.3333) results in 10

Conditional functions: to test an expression for TRUE or FALSE.

Function	Return Type	Description	Sample
ISNULL(Z)	Boolean	It returns TRUE if Z is NULL else FALSE	ISNULL('NULL') results TRUE
ISNOTNULL(Z)	Boolean	It returns TRUE if Z in not NULL else FALSE	ISNOTNULL('NULL') results FALSE
NVL(arg A, arg B)	STRING	It returns arg B if agr A is NULL else return arg A	NVL('NULL', 'Value is NULL) results in 'Value is NULL

Date functions:

Function	Return Type	Description	Sample
YEAR(string date)	INT	It returns year part of a date	YEAR('1992-01-01') return 1992
CURRENT_DATE	DATE	It returns the current date of query exection	CURRENT_DATE() return today's date
MONTHS_BETWEEN(date 1, date 2)	DOUBLE	It returns the number of months between date 1 and date 2	MONTHS_BETWEEN('1992-01-01','1993-01-01) return values as 12

Collection functions: to transform and retrieve a part of collection types like map, array, etc..

Function	Return Type	Description
SIZE(map<k,v>)	INT	It returns the total number of elements in map
SIZE(array<A>)	INT	It returns the total number of elements in array
ARRAY_CONTAINS (array<A>, value)	BOOLEAN	It returns TRUE if the array contains the mentioned value

User-defined functions

Developed in Java, can be integrated with Hive. Allows code reusability.

Types:

- UDF (user defined function): works on a single row, and produces a single row as output.
- User Defined Aggregate Functions (UDAF): Works on more than one row and produces one single output.
- User Defined Tabular Functions (UDTF): Works on one row and produces more than one output or rows. Used to split a single column into multiple values.

Steps to Create UDF

1. Open Eclipse IDE- Create a new Java project.

2. Add dependencies- Add Hadoop and Hive jar files to the project classpath.

3. Create Java class- Extend org.apache.hadoop.hive.ql.exec.UDF interface.

4. Create Jar- Package the project into a jar file.

5. Open Hive terminal- Add the jar to your current Hive location.

6. Create temporary function- Use the complete class name for the UDF created.

HIVE FILE FORMATS

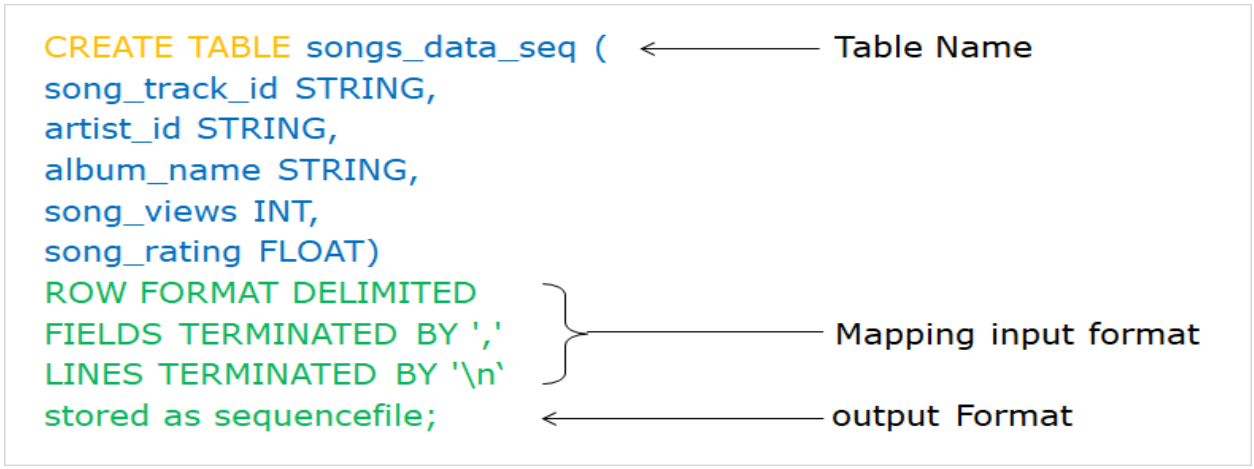
File format specifies how records are encoded in a file. Hive uses Hadoop's InputFormat APIs to read data from a variety of sources and OutputFormat APIs to write data to various sources.

Text file format(.csv/.tsv)

- Default file format for Hive
- Suitable for sharing data with other tools
- Can be viewed/edited manually
- Can be compressed by different compression formats like GZip, Snappy, LZO)

Sequence file format

- Stores data in machine understandable binary format
- Very useful for compressing data at block level and more compact than text files
- Allows sorting file data



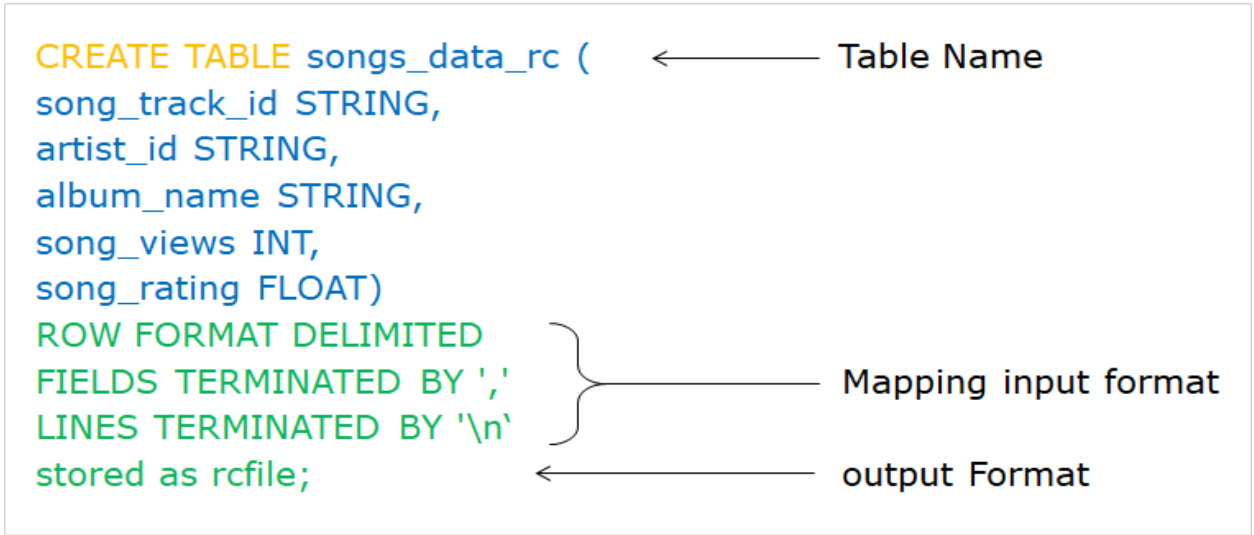
Verify input / output format by : DESCRIBE EXTENDED songs_data_seq;

To insert: INSERT OVERWRITE TABLE songs_data_seq; SELECT * FROM songs_data;

Both text file and sequence file formats follow row level storage format. They are not an optimal solution when querying one or two columns from the table. This problem can be solved using hybrid row columnar file formats like RC File, ORC File and similar other file formats.

RC(Row Columnar) File Format

- Splits data horizontally into row groups
- Stores grouped row data in columnar format
- Faster query results



ORC(Optimized Row Columnar) File Format

- Improved version of RC File format
- The data splits can be of larger size up to 256 MB and sequence split size of 1 MB
- Does not depend on meta store to recognize datatypes
- Supports statistical operations like MAX, MIN, SUM, COUNT on the columns
- Enables fast data load and query execution, efficient data storage and disk utilization

Avro file format

- Stores binary data in JSON format
- More compact and efficient for analysis
- Large files can be divided into smaller subsets making the execution much easier

Parquet file format

- Another columnar format similar to ORC file format
- Has a wider range of support for Hadoop ecosystem projects

Best Practices

Text file: Delimited data by special characters or parameters.

Sequence file: Small files with size less than the block size.

RC file: Efficient storage for data analytics operations.

ORC file: Optimized storage to reduce space and enhance performance.

SQOOP (SQL to Hadoop)

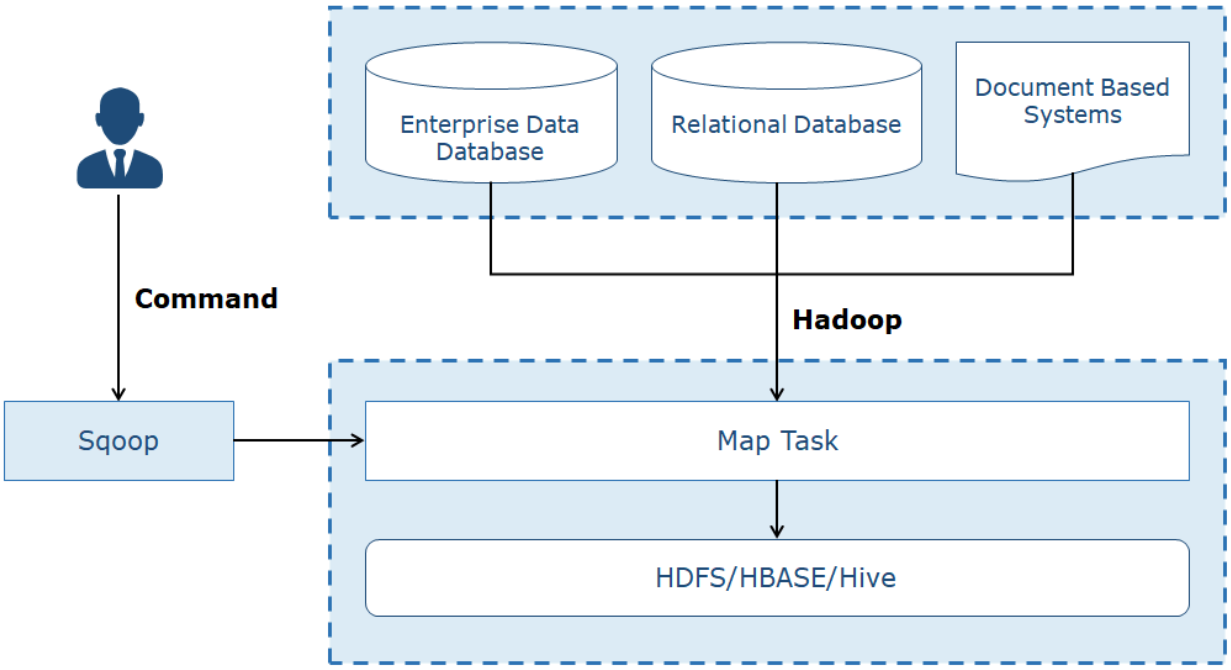
There is a need to move data from RDBMS to Hadoop to perform analysis, and to move processed data back to RDBMS.

Features:

- Used to transfer data between RDBMS (Oracle, MySQL etc) and Hadoop (HDFS, Hive, HBase, etc).
- Makes use of Mapreduce framework to import and export huge amounts of data, which provides parallel operation as well as fault tolerance.
- Migrates data directly to Hive or HBase.

- Provide full data migration, incremental data migration and connectors for all major RDBMS.
- Can be integrated with Oozie, which allows to schedule and automate import and export tasks.

SQOOP ARCHITECTURE

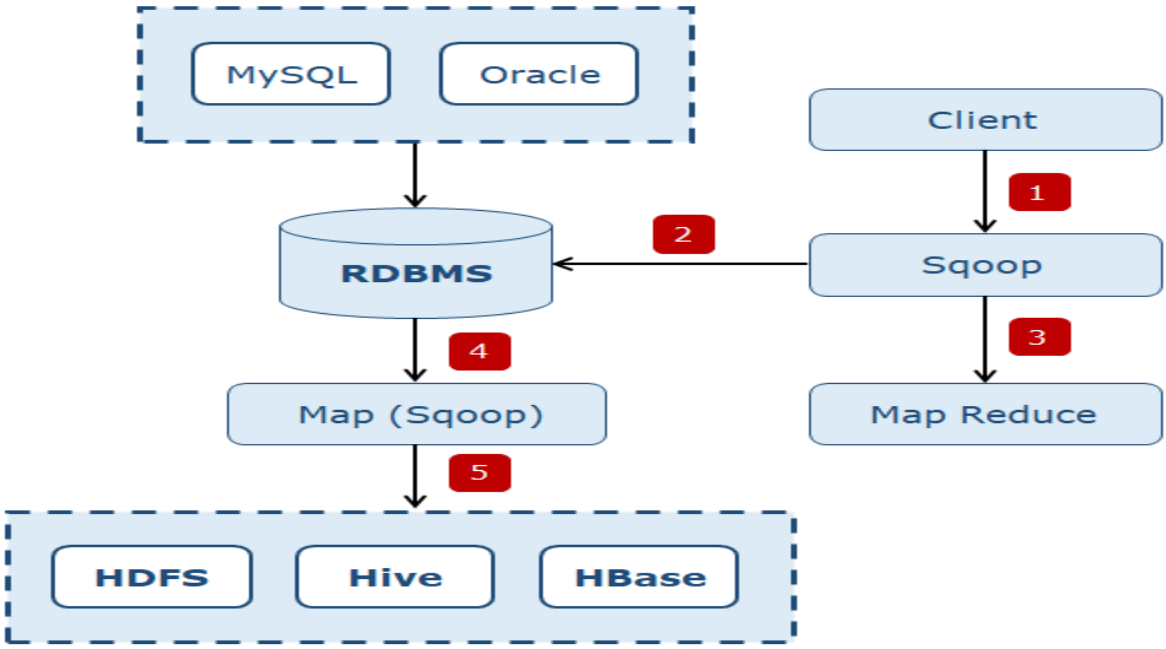


- Sqoop performs data migration between Hadoop and RDBMS/NoSQL.
- Individual tables are migrated to HDFS from RDBMS, where each row in the table becomes a record in the file.
- Data migration task gets divided into multiple sub tasks, where each one is handled by an individual map task.
- Every individual map task is responsible for transferring a slice of data. Collectively all map tasks migrate data in parallel in the most fault tolerant manner.
- Each record of data is handled in a safe manner as Sqoop uses the database metadata to infer the data types.

SQOOP DATA FLOW MODEL

-SQOOP IMPORT: FROM RDBMS/NoSQL TO HADOOP

All the Sqoop commands to be executed within the Hadoop environment ($ command prompt).



1. Run Import
2. Pull Meta Data
3. Launch Map Reduce Job
4. Pull data from database
5. Write to disks

Metadata retrieval

- Fetch table metadata: Number of columns, names, data types.
- Additional metadata: Partition information (depends on database).

Java class generation

- Query catalog: No data transfer yet.
- Generate java class: Compile using JDK and Hadoop libraries.

Data transfer

- MapReduce job: Connect to Hadoop cluster.
- Parallel transfer: Multiple mappers transfer data slices simultaneously.
- Direct transfer: Each mapper transfers data directly between database and Hadoop cluster.

```
sqoop import                                  <-- tool
--connect jdbc:mysql://blrkecetarhl01/products
--username root
-P
--table products
--num-mappers 1                               <-- configure the number of map tasks
--target-dir /user/etauser/product_mapper     <-- HDFS destination dir
```

Parameters:
--connect: takes jdbc url and connects to database
--username: authentication username to connect with database
--P: authentication password reads from console
--table: source table name to be imported
--num-mappers: decides the number of mappers to be launched; for degree of parallelism
--target-dir: imports data to the specified HDFS directory

```
--query 'select * from products WHERE productID > 1003 AND $CONDITIONS'
--split-by productID                          <-- used for slicing
```

--query: executes the SQL query and imports the results
--split-by: column used to split records
–where: retrieves records with the specified condition; –where “productID<1003”

To optimize import and export,

```
--table products                              <-- Enable compression
--direct                                      <-- use direct connectors
--compress
```

--direct: to use direct connectors provided by database
--compress: to enable compression

Import all tables,

```
sqoop import-all-tables                       <-- tool
--connect jdbc:mysql://blrkecetarhl01/products
--username root                               } tool-specific args
-P
```

Incremental import,

```
sqoop import                                              <-- tool
--connect jdbc:mysql://blrkecetarhl01/products
--username root
-P
--query 'select * from products WHERE $CONDITIONS'
--split-by productID                                      <-- used for slicing
--check-column productID
--incremental append                                      } incremental import arguments
--last-value 1004
--target-dir /user/etauser/product_incremental            <-- HDFS destination dir
```

Incremental import arguments

```
--check-column <col>     -> column to be examined
--incremental <mode>     -> determines which row are new
--last-value <value>     -> maximum value of the check column from earlier import
```

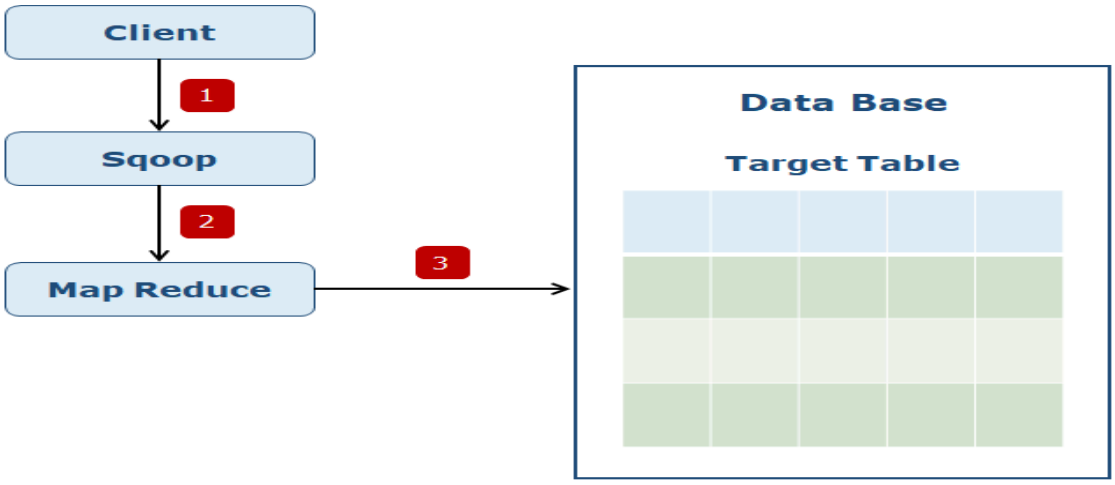
Import directly to Hive,

```
sqoop import                                              <-- tool
--connect jdbc:mysql://blrkecetarhl01/products
--username root
-P
--table products
--hive-import
--create-hive-table                                       } Hive arguments
--hive-table product_mysql
--target-dir /user/etauser/SqoopFS/product_mysql
```

Hive arguments

```
--hive-import                -> import table into hive
--create-hive-table          -> if set, then the job will fail if the
                                target hive tables already exist.
--hive-table <table-name>    -> sets the table name
```

-SQOOP EXPORT: FROM HADOOP TO RDBMS/NoSQ



1. Run Sqoop Export
2. Launch Map Reduce Job
3. Write to target table in Database using multiple mapper tasks

- Map tasks are part of a MapReduce job. They retrieve chunks of data from HDFS files in parallel, and export data to the destination table in the database.
- Multiple map tasks export data simultaneously. This parallel processing enhances performance.
- The destination table in the RDBMS must exist before executing the Sqoop command.

As the target table must exist in the RDBMS, create it.

```
CREATE TABLE IF NOT EXISTS products_export (   <-- Target table name
productID INT,
productCode CHAR(255),
name  VARCHAR(255),
quantity INT,
price VARCHAR(100),
PRIMARY KEY (productID));
```

Export command,

```
sqoop export                                   <-- tool
--connect jdbc:mysql://blrkecetarhl01/products
--username root                                <-- tool-specific args
-P
--table products_export
--export-dir products                          <-- export file
--batch                                        <-- method
```

--export-dir: exports data from the specified directory
--batch: mode of execution; to export in bulk

Staging tables

As Sqoop exports data in multiple transactions, there is a chance of failures leading to partial data being exported. Ensures fault-tolerant data export from HDFS to RDBMS.

Process:

1. Initial export: Data is first exported to a staging table.
2. Final export: If all map tasks succeed, data is moved to the main table in a new transaction.

Requirements:

- Space: Enough space for two copies of data.
- Staging table: Must exist and be empty, with the same structure as the main table.

Commands:

- Specify staging table: --staging-table
- Clear staging table: --clear-staging-table

Failure handling:

- Unique constraint violations cause job failure, inserting only partial data.

SPARK CORE

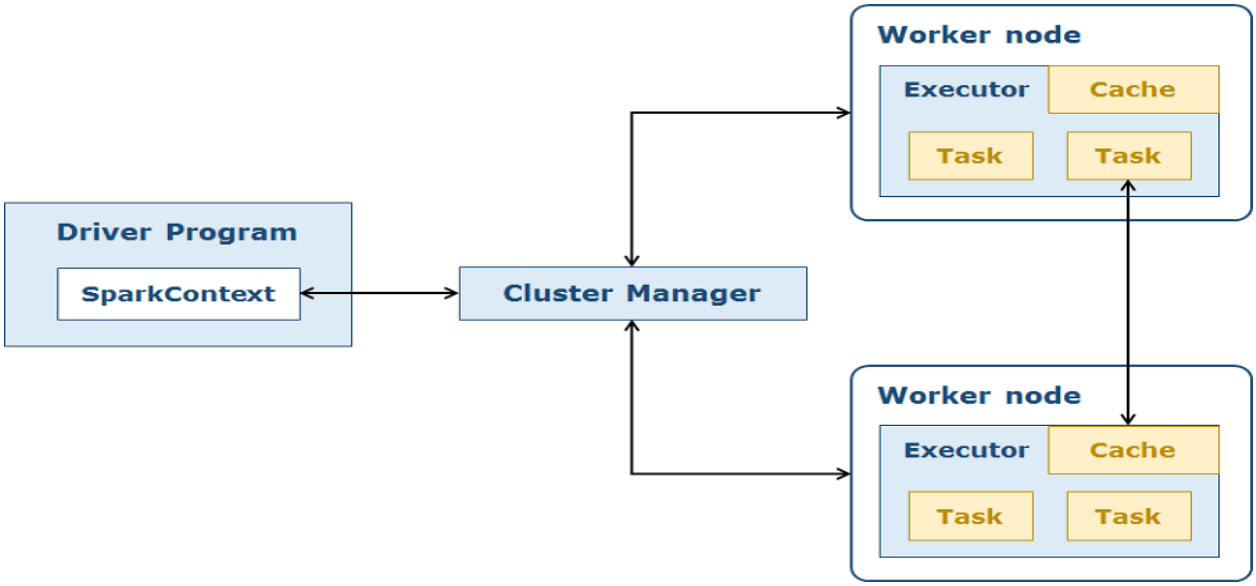
SPARK INTRO

From BD-201.

SPARK CORE ARCHITECTURE

Spark applications run as different sets of independent processes on Spark Cluster, where all processes are coordinated by **SparkContext** object created in the driver program.
SparkContext object acts as an entry point for all operations in Spark application.
Driver program is where,

- SparkContext gets created
- Program gets converted into lineage execution graph
- Tasks get scheduled and monitored



- SparkContext connects to the respective cluster manager (YARN in hadoop 2.x, Mesos etc) where the cluster manager in turn allocates resources to Spark jobs.
- Spark creates executors on slave nodes in the cluster. Executors are processes meant to run computations against the data blocks spread across HDFS.
- SparkContext then sends across the tasks to all the executors to run it in the respective nodes. Task is a unit of execution that computes the data in each Worker node.
- Worker nodes are the servers where actual data to be processed is stored.

RESILIENT DISTRIBUTED DATASET (RDD)

Spark's fundamental data structure used for in-memory computations.
Immutable collection of records distributed in the form of partitions across the cluster nodes. Computations happen in parallel on these partitions.

RDD Lineage graph

In Spark, any application execution leads to a Directed Acyclic Graph (DAG) creation.
DAG is a special graph containing vertices and edges with no direct cycles. In the RDD lineage graph, vertices represent RDDs and edges are the operations applied on RDDs.
DAG indicates what transformations to be executed once an action statement is encountered. Every Spark transformation statement leads to an RDD creation whereas an action statement returns a final value.

RDD transformations

Transformation is a function that processes existing RDD and produces new RDD as an output. Input RDD can not be modified. Applying transformations will build the RDD lineage graph which is used while executing the application.
Lazy in nature and get executed only when an action statement is encountered in the program.

- map(function) - Creates a new RDD by passing each record in the source RDD through a user defined function.
- flatMap(function) - Similar operation as map() but each input item can be mapped to 0, 1 or more output items.
- filter(function) - Creates a new RDD by filtering the records of the source RDD on which function returns true.
- repartition(new number of partitions) - Modifies the degree of parallelism by changing the number of partitions.
- union(other RDD) - New RDD will be created by union of two RDDs.
- reduceByKey(function) - When called on a RDD of (K, V) pairs, returns a new RDD of (K,V) pairs where the values for each key are aggregated using the given reduce function.
- join(other RDD, [numTasks]) - When called on two RDDs of (K, V) and (K, W) pairs, returns a new RDD of (K, (V, W)) pairs with all pairs of elements for each key.

RDD Actions

RDD operations which do not produce RDD as an output. These operations finally produce a value which can be saved on the disk.
Lazy evaluation pattern. Until an action is encountered in the program, transformations are not executed. These operations can really create all the RDDs in the lineage graph. Final output is sent back by executors to the driver program.

- reduce(func): aggregates the elements of the dataset based on the function logic provided. This operation can be applied only on commutative and associative operations.
- collect(): return all the elements of RDD as an array at the driver program.
- count(): returns the number of elements in RDD.
- first(): returns the first element of RDD.
- take(n): returns first n elements of RDD as an array.
- saveAsTextFile(path): saves RDD content as a text file in a provided path in the local file system or HDFS.

Working with RDDs

Filter erroneous routers

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark.SparkConf
object SparkWordCount {
def main(args: Array[String]) {
//sc refers to the built-in spark context
val sc = new SparkContext(new SparkConf().setAppName("Spark Scala Application"))
//Load dataset into RDD
  val rdd = sc.textFile("HDFS file path"); } }
```

To load array or list data,

method called parallelize(input data,no. of partitions)

```
val myarray = Array(1,2,3,4,5);
val rdd = sc.parallelize(myarray,5);
```

Note: also, sc.textFile("File Path", no of partitions)

Filter fields,

```
rdd.map(_.split("\t")).map(r =>r(8)+","+r(9));//split the fields and concatenate r(8) & r(9) using ,

//Filtering the ERROR records and count the router specific errors
val rdd3 = rdd2. filter(_.startsWith(“ERROR”)) //Filter records starting with errors
val R1Errs = rdd3.map(_.contains(“RTR1”)).count() //Count Rtr1 records
val R2Errs = rdd3.map(_.contains(“RTR2”)).count() // Count Rtr2 records
val outarray = Array(“Router 1 contains ”+R1Errs,”Router 2 contains ”+R2Errs); //Collate all
router output to an array
```

Paired RDDs

Aggregation is applied using **reduceByKey(function)** transformation that executes the logic defined by function against values of each key.

```
 val lines = sc.textFile(“/hdfspath/routerLog.tsv ")
 val pairs = lines.map(record => (record, 1))
 val counts = pairs.reduceByKey((a, b) => a + b)
 counts.collect.foreach(println)
 counts.collect.saveAsTextFile(“/hdfspath/myuniquedirectory”)
```

SPARK CORE JOINS

Provides join() function which can be used to join two paired RDDs based on the same key.

Function	Purpose	Usage
join()	Perform an inner join between two RDDs	rdd.join(other)
rightOuterJoin()	• Perform a join between two RDDs • Key must be present in the first RDD	rdd.rightOuterJoin(other)
leftOuterJoin()	• Perform a join between two RDDs • Key must be present in the other RDD	rdd.leftOuterJoin(other)

Implementation steps to join

1. Create Case classes representing schema of datasets

case class RouterLocation(rid:Int,name:String,location:String);
case class RouterPurchase(rid:Int,date:String,pmemory:Long,smemory:Long,cost:String);

2. Generate <K,V> pairs using case class object

Datasets are loaded as RDDs.
Paired RDDs <K, V> are created where K = common column in both RDDs, V = value part which contains complete record within case class object.

```
//Load dataset and generate Rid(common field),object
val locRDD = sc.textFile("RouterLocation.tsv").map(_.split("\t")).map(r => (r(0),
RouterLocation(r(0).toInt,r(1),r(2))))
val purRDD = sc.textFile("RouterPurchase.tsv").map(_.split("\t")).map(r =>
(r(0),RouterPurchase(r(0).toInt,r(1),r(2).toLong,r(3).toLong,r(4))))
```

3. Apply join function

```
//Join locRDD with purRDD using join()
locRDD.join(purRDD).collect()
```

SPARK PERSISTENCE

RDD Persistence

RDDs can be cached in memory via cache() or persist() method leading to performance optimization. Cached RDDs are fault tolerant in nature.

Persistence Options	Explanation
MEMORY_ONLY	Persists RDD only in RAM, not on disk
MEMORY_AND_DISK	Persists RDD in RAM, if RAM is full then persist it on disk
MEMORY_ONLY_SER	Persist RDD as serialized Java objects only on RAM
MEMORY_AND_DISK_SER	Persist RDD as serialized Java objects only on RAM, if RAM is full then use disk
DISK_ONLY	Store the RDD partitions only on disk
MEMORY_ONLY_2, MEMORY_AND_DISK_2	Same as the levels above, but replicate each partition on two cluster nodes

Note: RDD.cache() method is equivalent to RDD.persist(MEMORY_ONLY).

Unpersisting RDD

Spark automatically monitors cache usage on each node and drops out old data partitions in a least-recently-used (LRU) manner.
To manually remove an RDD, use RDD.unpersist() method.

Shared variables

As per Spark architecture, variables defined in driver code will lose their scope as executors run the code across machines separately.
Solution: Create Shared variables as they retain their scope across executors running Spark tasks. Shared variables inside driver code solves the issue by keeping variables on stack rather on heap.

Types:

- ➔ Accumulators
  - ◆ Provide a simple way of aggregating values in slave nodes and sending them back to the driver program.
  - ◆ Used to count events that occur during job execution and debugging purposes, and implement counters (as in MapReduce).

- Limitations: if the executor fails for any reason, there'll be an inconsistent value for count or sum as it executes from beginning again.

```
val rdd = sc.textFile("/HDFSPath");
//Create and initialize the accumulator to count blank lines in  the file.
val blankLinesCounter = sc.accumulator(0);
//verify each line and increment the counter
rdd.foreach { line =>
if(line.length() == 0) blankLinesCounter  += 1 }
println("Empty Lines: " + blankLinesCounter )
```

➔ Broadcast variables
- Used to send large values over the cluster, compared to an accumulator.
- Cached on each machine rather than moving a copy of it with tasks.
- Read-only in nature.

```
//Creating a broadcast variable with simple array of integers
val broadcastVar = sc.broadcast(Array(100, 200, 300))
//Retrieving the value of a broadcast variable
broadcastVar.value
```

1:05 pm

SPARK SQL

SPARK CORE CHALLANGES

No schema or table like structure for advanced aggregations.
Use of indices (_._1) instead of field names complicates understanding.
Code customization is difficult with schema changes.
Aggregate functions have to be applied individually.

SPARK SQL OVERVIEW

One of the Spark ecosystem components used for structured data processing. It provides a structure/schema for the dataset and helps to perform computations using SQL queries.

Features

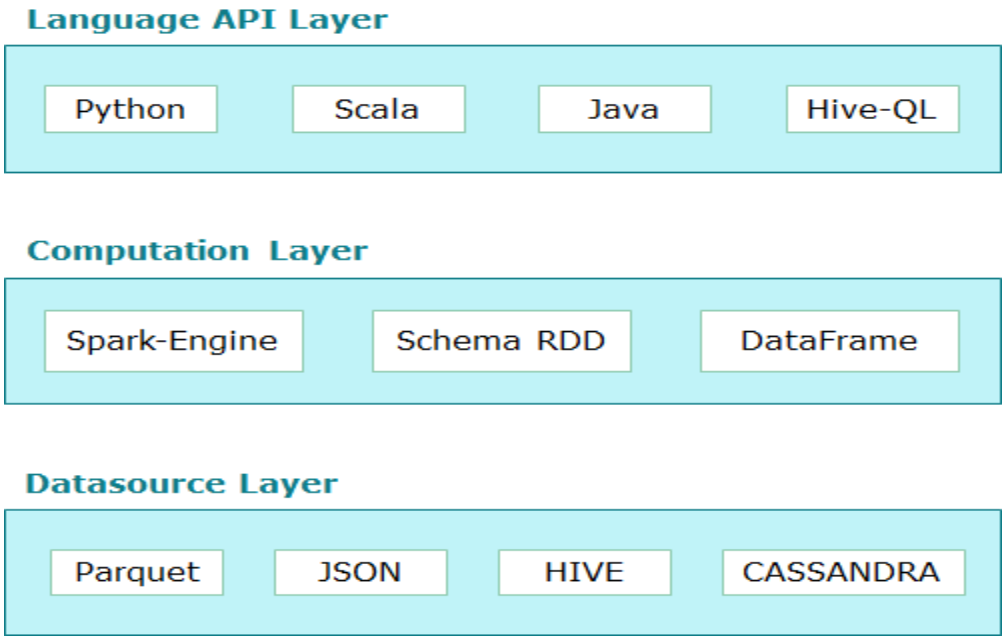
- Ability to write queries for data analysis
- Ability to analyze data available in Hive tables
- Spark SQL queries run nearly 100 times faster than Hive queries
- Supports different file formats like Parquet, JSON, XML, Text, CSV/TSV etc

Programmatic Components

Provides schema to the dataset and runs queries on top of it.

- DataFrame (Available from Spark 1.3 version)
- Dataset (Available from Spark 1.6 version)

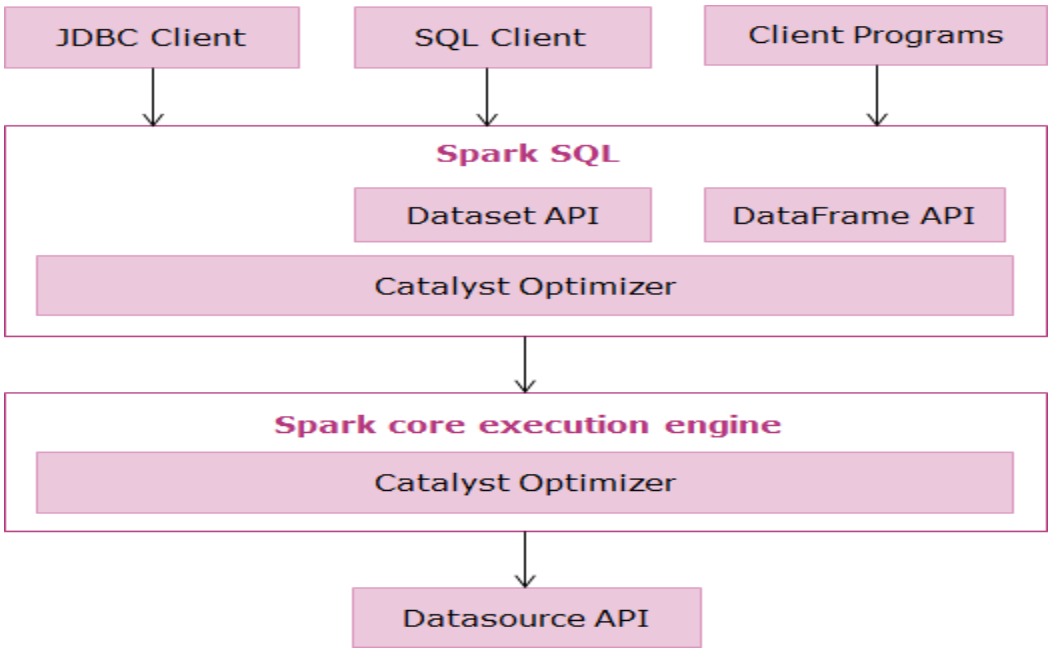
SPARK SQL ARCHITECTURE



Data source layer: refers to input file formats like Parquet, JSON, CSV, Hadoop files and JDBC data sources for analysis.
Computation layer: provides Schema RDD (older version), DataFrame and Dataset API for structuring and querying data. Spark SQL query gets parsed and executed in this layer.

Language API layer: lists out different programming languages to access data in DataFrames and datasets via JDBC/ODBC and console clients.

SPARK SQL WORKFLOW



Execution Flow

1. Query execution: JDBC/ODBC and console clients execute a Spark SQL query using DataFrame/dataset API.
2. Data processing: Data in DataFrame/dataset is processed and optimized using the catalyst optimizer.
3. Optimization: Catalyst optimizer converts the query to a physical plan, optimizes, and compiles parts of queries to Java byte code.
4. RDD construction: The optimized byte code constructs RDDs as per the query plan, executed within Spark engine as a DAG.
5. Output persistence: The output is persisted in a file format supported by Datasource API.

Role of catalyst optimizer:

- Constructs a logical plan for the query.
- Converts the logical plan into a physical plan for execution.

Spark SQLs integration with other Spark components

Structured Streaming

- Definition: Scalable and fault-tolerant stream processing engine.
- Built on: Spark SQL engine.
- Availability: From Spark 2.3.0 version.
- APIs: DataFrame/Dataset APIs in Scala, Java, Python, and R.
- Usage: Streaming data analysis and batch processing.

Spark SQL with Machine Learning

- Package: Spark's machine learning package (spark.mllib from 2.3.0 version).
- APIs: Supports both RDD-based and DataFrame-based APIs.
- Advantages:
    - DataFrame API is more user-friendly.
    - Leverages Spark SQL features like DataFrame queries and catalyst optimizer.

Spark SQL with Graphx

- Integration: GraphFrames integrates pattern matching and graph algorithms.
- Features:
    - Graph-aware query planner for faster queries.
    - Supports analysis on HDFS data and NoSQL databases like HBase and Cassandra.

SQLContext

Entry point to all Spark SQL operations.
Created on top of SparkContext object.
Provides various methods to create DataFrame from different file formats.
Allows to write SQL queries on top of a DataFrame.

```
val sqlContext = new org.apache.spark.sql.SQLContext(sc)
import sqlContext.implicits._ // to access methods that implicitly convert RDD to a dataframe
```

SPARK SQL PROGRAMMING COMPONENTS

The first step in data processing using Spark SQL is to load a normal Spark RDD into either a DataFrame or Dataset. Both are special API classes of Spark SQL.
They help to load a Spark RDD, provide a schema to it with the help of a case class, then register it as a table and further execute Spark SQL queries against the table.

DataFrame	Dataset
DataFrame is a distributed collection of data organized in to named columns	Dataset is an extended version of a DataFrame which provides a type-safe programming interface
Introduced in 1.3 version	Introduced in 1.6 version
Limited support for lambda expressions	Additional support for lambda expressions
DataFrame API available in Scala, Java, Python and R	Dataset API available in Scala and Java only. No support for python
DataFrame can be constructed from existing RDDs, Hive tables and data files	Dataset can also be constructed from existing RDDs, Hive tables and data files
Allows to work with structured data	Support for both structured and unstructured data

DataFrame API

1. Create a SparkContext object and load the dataset using the SparkContext object's textFile() method

```
val Car_Info = sc.textFile("/HDFSpath/ArisconnCars.txt");
//dataset's first record contains schema details which needs to be ignored while processing
val header = Car_Info.first()
```

2. Create a SQLContext object using SQLContext class

```
val sqlContext = new org.apache.spark.sql.SQLContext(sc);
```

3. Create a DataFrame and load a normal RDD in to the DataFrame using Case class

```
case class Cars(sensorid: String, carid: String, latitude:Double, longitude:Double,
engine_speed:Int, accelerator_pedal_position:Int, vehicle_speed:Int, torque_at_transmission:Int,
fuel_level:Double, TypeOfMessage:String, timestamp:Double)
/*Creation of a Spark SQL DataFrame by delimiting the fields, and loading them as case class
properties. Note: "toInt", "toDouble" are Scala methods for converting text fields in to
numerical*/
val DF = Car_Info.filter(c => c!=header).map(_.split(",")).map(c => Cars(c(0), c(1),
c(2).toDouble, c(3).toDouble, c(4).toInt, c(5).toInt, c(6).toInt, c(7).toInt, c(8).toDouble, c(9),
c(10).toDouble)).toDF()
```

4. Register the DataFrame as a normal table using registerTempTable() method

```
DF.registerTempTable("cars");
```

5. Write and execute Spark SQL queries against the table using SQLContext object's sql() method

```
//Query using sql method to filter corrupted records, consider only valid data for further analysis
val valrecords= sqlContext.sql("SELECT sensorid, carid, latitude, longitude, vehicle_speed,
TypeOfMessage FROM cars WHERE sensorid Like 'SEN_%' AND carid Like 'CAR_%' AND
sensorid!='sensorID'");
valrecords.show() //display the dataframe records on console
```

Function Name	Purpose	Example
show()	Show the contents of a DataFrame "df"	df.show();
select()	Select the specified fields or functions from DataFrame "df"	df.select("name").show();
filter()	Select only the rows meeting the criteria from "df"	df.filter(df.col("age").gt(19)).show();
groupBy()	Group together on a column, to be followed by aggregations like min(), max(),mean() etc.	df.groupBy("age").count().show();

Dataset

It is a specialized Dataframe where elements map to specific JVM object types instead of row class. Can be used instead of Dataframes for Spark SQL analysis, supports lambda expressions and offers high performance. Data is stored in the encoded form and a special encoder does the job of mapping the dataset with the schema.

Interoperability:

Convert dataset to dataframe - ds.toDF() method

Dataframe to dataset - df.as[element type]
case class ArisconData(name: String, age: Int)
val ds: Dataset[ArisconData] = df.as[ArisconData]
(ds is the dataset created from a DataFrame with the help of a type class)

Dataset API

```
val sqlContext = new org.apache.spark.sql.SQLContext(sc);
val Cust_Info = sc.textFile("/HDFSPath/Customer.txt");
case class Customer(custid: String,firstname: String,lastname:String,age:Int,profession:String);
//Creating a Dataset using toDS() method
val DS = Cust_Info.map(_.split(",")).map(c => Customer(c(0),c(1),c(2),c(3).toInt,c(4)).toDS();
//Creating and registering the Dataset as a temporary view
DS.createOrReplaceTempView("customer");
//Spark SQL query to compute Profession, Average age of that profession
val pilotavg= sqlContext.sql("SELECT profession,AVG(age) FROM customer group by
profession");
pilotavg.show();
```

WORKING WITH DIFFERENT FILE FORMATS

Processing Paraquet files

It is a column file format; preserves schema of data while storing it; provides better performance while storing and processing data;
Spark SQL offers API functions to read/write parquet files.

```
//Store the results of the DataFrame as a parquet file in an HDFS directory
valrecords.write.parquet("/HDFSPath/CarsParquetData")
```

```
//read data from parquet file and create a DataFrame
val CarsDF = sqlContext.read.parquet("/HDFSPath/CarsParquetData")
//display the DataFrame content
CarsDF.show()
```

Processing JSON files

Schema is inferred automatically. Spark SQL runs against it.

```
//Creation of SQLContext
object val sqlContext = new org.apache.spark.sql.SQLContext(sc);
//Spark SQL provides read.json() method to load JSON files as DataFrames directly
val cars = sqlContext.read.json("/HDFSPath/Cars.json");
//Schema of JSON files shall be printed using printSchema() method
cars.printSchema();
//Creation of a temporary table
cars.registerTempTable("carstable");
//Spark SQL query against carstable
val op = sqlContext.sql("SELECT * from carstable");
//show() method to print output
op.show();
```

Processing Avro files

Avro file format provides high performance in read and write, and optimizes storage in compressed format. Uses JSON to define data types and prototypes; serializes data into binary format using JSON.

Spark SQL supports reading and writing Avro files using spark-avro library.

Library provides below methods:

- read.avro("inputdir") - create DataFrame from Avro file
- write.avro("outputdir") - Save a DataFrame into Avro file

```
import com.databricks.spark.avro._
//Creation of SQLContext object
val sqlContext = new org.apache.spark.sql.SQLContext(sc);
//Spark SQL provides read.avro() method to load Avro files as DataFrames directly
val cars = sqlContext.read.avro("/HDFS/Cars.avro");
//Schema of avro files shall be printed using printSchema() method
cars.printSchema();
//Creation of a temporary table
cars.registerTempTable("carstable");
```

```
//Spark SQL query against carstable
val op = sqlContext.sql("SELECT * from carstable");
//show() method to print output
op.show();
```

SPARK SQL ON HIVE

When Hive queries are executed directly in the Hive environment, they are converted to a series of MapReduce jobs and get executed. This leads to performance issues.
Solution:

- Read the data from Hive tables to Spark SQL environment and process
- Hive becomes a data source to Spark SQL
- Spark engine is used for processing instead of MapReduce
- Additional installation steps are required to integrate Spark SQL and HivE

SPARK SQL QUERIES ON HIVE TABLES

1. Load the parquet file as a dataframe

```
val sqlContext = new org.apache.spark.sql.SQLContext(sc)
val carsparq = sqlContext.read.parquet("/HDFS/CarsParq.parquet")
```

2. Save dataframe into Hive tables

```
//write.option method used to specify the path for the Hive table and saveAsTable method creates
//a Hive table in Hive metastore
carsparq.write.option("path",'/HDFS/jaicarsparq').saveAsTable("SparkHiveCarsTable")
```

3. Spark SQL queries on Hive

```
//Creation of HiveContext object for Spark to connect to Hive
val hContext = new org.apache.spark.sql.hive.HiveContext(sc);
//Spark SQL query on a hive table to compute the top 10 vehicles which were running on
maximum speed
val CarHiveData = hContext.sql("select carid,latitude,longitude,MAX(vehicle_speed) from
SparkHiveCarsTable GROUP BY carid limit 10")
//Output of the query to be displayed
CarHiveData.show();
```

SPARK STREAMING

For processing real time data.
Need for it:

- Data can be analyzed immediately as and when received. Quick insights helps to improve business in an agile manner
- Provides complete solution to consume any amount of logs from different data sources and process at high speed
- Supports analytics that can be performed on data collected for a given window period
- Provides fault-tolerant stream processing of real-time data
- Supports data checkpointing in order to recover from failures

Features:

- It is an open source, distributed framework that provides an efficient way to deal with real-time data processing
- It provides set of programming APIs to collect and process live continual data
- Spark Streaming applications can be written using Java, Scala or Python
- Spark Core, SQL and Machine learning algorithms can be applied on the continuous streams
- Support for structured data streaming where data streams can be treated as tables

